

Source Code

Below is the complete source code combining all the classes and functionalities from the exercises above into a single C# file. This example is structured as a Console Application for demonstration purposes.

```
using System;
using System.Collections.Generic;
using System.Linq;

// Exercise 1: Encapsulation - Book Class
public class Book : IPrintable
{
    public string ISBN { get; set; }
    public string Title { get; set; }
    public string Author { get; set; }

    private int year;
    public int Year
    {
        get { return year; }
        set
        {
            if (value >= 0)
                year = value;
            else
                throw new ArgumentException("Year cannot be negative.");
        }
    }

    private int copiesAvailable;
    public int CopiesAvailable
    {
        get { return copiesAvailable; }
        set
        {
            if (value >= 0)
                copiesAvailable = value;
            else
                throw new ArgumentException("CopiesAvailable cannot be negative.");
        }
    }

    public void DisplayInfo()
    {
        Console.WriteLine($"ISBN: {ISBN}, Title: {Title}, Author: {Author}, Year: {Year}, Copies Available: {CopiesAvailable}");
    }

    public void PrintDetails()
    {
        DisplayInfo();
    }
}

// Exercise 2: Inheritance - Member and PremiumMember Classes
public class Member : IPrintable, IMemberActions
```

```

{
    public string MemberID { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }

    public virtual void DisplayInfo()
    {
        Console.WriteLine($"MemberID: {MemberID}, Name: {Name}, Email:
{Email}");
    }

    public virtual void BorrowBook(Book book)
    {
        if (book.CopiesAvailable > 0)
        {
            book.CopiesAvailable--;
            Console.WriteLine($"{{Name}} borrowed '{{book.Title}}'.");
        }
        else
        {
            Console.WriteLine($"No copies available for '{{book.Title}}'.");
        }
    }

    public virtual void ReturnBook(Book book)
    {
        book.CopiesAvailable++;
        Console.WriteLine($"{{Name}} returned '{{book.Title}}'.");
    }

    public virtual void PrintDetails()
    {
        DisplayInfo();
    }
}

public class PremiumMember : Member
{
    public DateTime MembershipExpiry { get; set; }
    public int MaxBooksAllowed { get; set; }

    public override void BorrowBook(Book book)
    {
        if (book.CopiesAvailable > 0 && (MaxBooksAllowed > 0))
        {
            book.CopiesAvailable--;
            MaxBooksAllowed--;
            Console.WriteLine($"{{Name}} (Premium) borrowed '{{book.Title}}'.");
        }
        else
        {
            Console.WriteLine($"Cannot borrow '{{book.Title}}'.");
        }
    }

    public override void DisplayInfo()
    {

```

```

        base.DisplayInfo();
        Console.WriteLine($"Membership Expiry:
{MembershipExpiry.ToShortDateString()}, Max Books Allowed:
{MaxBooksAllowed}");
    }

    public override void PrintDetails()
    {
        base.PrintDetails();
        Console.WriteLine($"Membership Expiry:
{MembershipExpiry.ToShortDateString()}, Max Books Allowed:
{MaxBooksAllowed}");
    }
}

// Exercise 3: Abstraction - Transaction and Derived Classes
public abstract class Transaction
{
    public string TransactionID { get; set; }
    public DateTime TransactionDate { get; set; }
    public Member Member { get; set; }

    public abstract void Execute();
}

public class BorrowTransaction : Transaction
{
    public Book BookBorrowed { get; set; }

    public override void Execute()
    {
        if (BookBorrowed.CopiesAvailable > 0)
        {
            BookBorrowed.CopiesAvailable--;
            Console.WriteLine($"{Member.Name} has borrowed
'{BookBorrowed.Title}'.");
        }
        else
        {
            Console.WriteLine($"No copies available for
'{BookBorrowed.Title}'.");
        }
    }
}

public class ReturnTransaction : Transaction
{
    public Book BookReturned { get; set; }

    public override void Execute()
    {
        BookReturned.CopiesAvailable++;
        Console.WriteLine($"{Member.Name} has returned
'{BookReturned.Title}'.");
    }
}

```

```

// Exercise 5: Interfaces - IPrintable and IMemberActions
public interface IPrintable
{
    void PrintDetails();
}

public interface IMemberActions
{
    void BorrowBook(Book book);
    void ReturnBook(Book book);
}

// Exercise 7: Overloading and Overriding - NotificationService Classes
public class NotificationService
{
    public virtual void SendNotification(string message)
    {
        Console.WriteLine($"Notification: {message}");
    }

    public void SendNotification(string message, string recipient)
    {
        Console.WriteLine($"Notification to {recipient}: {message}");
    }

    public void SendNotification(string message, List<string> recipients)
    {
        foreach (var recipient in recipients)
        {
            Console.WriteLine($"Notification to {recipient}: {message}");
        }
    }
}

public class AdvancedNotificationService : NotificationService
{
    public override void SendNotification(string message)
    {
        Console.WriteLine($"[{DateTime.Now}] Advanced Notification: {message}");
    }
}

// Exercise 10: Delegates and Events - NotificationService Classes
public class NotificationServiceHandler
{
    public void SendBorrowNotification(Book book, Member member)
    {
        Console.WriteLine($"Notification: {member.Name} has borrowed '{book.Title}'");
    }

    public void LogBorrowActivity(Book book, Member member)
    {
        Console.WriteLine($"Log: {member.Name} borrowed '{book.Title}' on {DateTime.Now}");
    }
}

```

```

    }
}

// Exercise 9: Difference Between Class and Record - BookClass vs BookRecord
public class BookClass
{
    public string ISBN { get; set; }
    public string Title { get; set; }
    public string Author { get; set; }

    public BookClass(string isbn, string title, string author)
    {
        ISBN = isbn;
        Title = title;
        Author = author;
    }
}

public record BookRecord(string ISBN, string Title, string Author);

// Exercise 4: Polymorphism - Handling Transactions
public class TransactionHandler
{
    public void HandleTransactions()
    {
        List<Transaction> transactions = new List<Transaction>();

        // Create members
        Member member1 = new Member { MemberID = "M001", Name = "Alice", Email
= "alice@example.com" };
        PremiumMember member2 = new PremiumMember { MemberID = "M002", Name
= "Bob", Email = "bob@example.com", MembershipExpiry =
DateTime.Now.AddYears(1), MaxBooksAllowed = 10 };

        // Create books
        Book book1 = new Book { ISBN = "ISBN001", Title = "C# Programming",
Author = "Author A", Year = 2020, CopiesAvailable = 5 };
        Book book2 = new Book { ISBN = "ISBN002", Title = "OOP Concepts", Author
= "Author B", Year = 2018, CopiesAvailable = 2 };

        // Create borrow and return transactions
        BorrowTransaction borrow1 = new BorrowTransaction
        {
            TransactionID = "T001",
            TransactionDate = DateTime.Now,
            Member = member1,
            BookBorrowed = book1
        };

        ReturnTransaction return1 = new ReturnTransaction
        {
            TransactionID = "T002",
            TransactionDate = DateTime.Now,
            Member = member1,
            BookReturned = book1
        };
    }
}

```

```

        transactions.Add(borrow1);
        transactions.Add(return1);

        // Execute transactions
        foreach (var transaction in transactions)
        {
            transaction.Execute();
        }
    }
}

// Exercise 6: Constructors - Library Class
public class Library
{
    public string LibraryName { get; set; }
    public List<Book> Books { get; set; }
    public List<Member> Members { get; set; }

    // Event when a book is borrowed
    public event Action<Book, Member> OnBookBorrowed;

    public Library(string libraryName)
    {
        LibraryName = libraryName;
        Books = new List<Book>();
        Members = new List<Member>();
    }

    public void AddBook(Book book)
    {
        Books.Add(book);
    }

    public void AddMember(Member member)
    {
        Members.Add(member);
    }

    public void BorrowBook(string isbn, string memberId)
    {
        var book = Books.FirstOrDefault(b => b.ISBN == isbn);
        var member = Members.FirstOrDefault(m => m.MemberID == memberId);

        if (book != null && member != null && book.CopiesAvailable > 0)
        {
            member.BorrowBook(book);
            OnBookBorrowed?.Invoke(book, member);
        }
        else
        {
            Console.WriteLine("Cannot borrow the book.");
        }
    }

    public void ReturnBook(string isbn, string memberId)

```

```

    {
        var book = Books.FirstOrDefault(b => b.ISBN == isbn);
        var member = Members.FirstOrDefault(m => m.MemberID == memberId);

        if (book != null && member != null)
        {
            member.ReturnBook(book);
        }
        else
        {
            Console.WriteLine("Cannot return the book.");
        }
    }
}

// Exercise 8: Properties with Access Modifiers - LibraryCard Class
public class LibraryCard
{
    public string CardNumber { get; private set; }
    public Member Owner { get; set; }
    public DateTime IssueDate { get; private set; }

    public LibraryCard(string cardNumber, Member owner)
    {
        CardNumber = cardNumber;
        Owner = owner;
        IssueDate = DateTime.Now;
    }

    public void RenewCard()
    {
        IssueDate = DateTime.Now;
        Console.WriteLine($"Library card {CardNumber} has been renewed on {IssueDate.ToShortDateString()}.");
    }

    public void PrintCardDetails()
    {
        Console.WriteLine($"Card Number: {CardNumber}, Owner: {Owner.Name}, Issue Date: {IssueDate.ToShortDateString()}");
    }
}

// Main Program to Demonstrate Functionality
public class Program
{
    public static void Main(string[] args)
    {
        // Initialize Library
        Library library = new Library("Central Library");

        // Add Books
        library.AddBook(new Book { ISBN = "ISBN001", Title = "C# Programming", Author = "Author A", Year = 2020, CopiesAvailable = 5 });
        library.AddBook(new Book { ISBN = "ISBN002", Title = "OOP Concepts", Author = "Author B", Year = 2018, CopiesAvailable = 2 });
    }
}

```

```

        // Add Members
        Member member1 = new Member { MemberID = "M001", Name = "Alice", Email
= "alice@example.com" };
        PremiumMember member2 = new PremiumMember { MemberID = "M002", Name
= "Bob", Email = "bob@example.com", MembershipExpiry =
DateTime.Now.AddYears(1), MaxBooksAllowed = 10 };
        library.AddMember(member1);
        library.AddMember(member2);

        // Create LibraryCard
        LibraryCard card1 = new LibraryCard("C001", member1);
        LibraryCard card2 = new LibraryCard("C002", member2);

        card1.PrintCardDetails();
        card2.PrintCardDetails();

        // Renew Library Card
        card1.RenewCard();

        // Create and Handle Transactions
        TransactionHandler transactionHandler = new TransactionHandler();
        transactionHandler.HandleTransactions();

        // Setup Notification Service
        NotificationServiceHandler notificationHandler = new
NotificationServiceHandler();
        library.OnBookBorrowed +=
notificationHandler.SendBorrowNotification;
        library.OnBookBorrowed += notificationHandler.LogBorrowActivity;

        // Borrow a Book
        library.BorrowBook("ISBN001", "M001");

        // Demonstrate Overloading and Overriding
        NotificationService basicService = new NotificationService();
        AdvancedNotificationService advancedService = new
AdvancedNotificationService();

        // Call overloaded methods
        basicService.SendNotification("Welcome to the library!");
        basicService.SendNotification("Your book is due tomorrow.", "Alice");
        basicService.SendNotification("Library meeting at 5 PM.", new
List<string> { "Bob", "Charlie" });

        // Call overridden method
        advancedService.SendNotification("System maintenance tonight.");

        // Compare Class and Record
        BookClass bookClass1 = new BookClass("ISBN123", "C# Basics", "Author
A");
        BookClass bookClass2 = new BookClass("ISBN123", "C# Basics", "Author
A");

        BookRecord bookRecord1 = new BookRecord("ISBN123", "C# Basics",
"Author A");

```



```

        BookRecord bookRecord2 = new BookRecord("ISBN123", "C# Basics",
"Author A");

        // Compare using ==
        Console.WriteLine(bookClass1 == bookClass2); // False
        Console.WriteLine(bookRecord1 == bookRecord2); // True

        // Use with to create a copy
        var bookRecord3 = bookRecord1 with { Title = "Advanced C#" };
        Console.WriteLine(bookRecord3); // BookRecord { ISBN = ISBN123, Title
= Advanced C#, Author = Author A }
    }
}
=====

```

Exercise 1 – Create the User, Role, and UserRole Models and Load from Database

Question

Create the *User*, *Role*, and *UserRole* models.

Define a many-to-many relationship between *User* and *Role*.

Load data from the database using *AppDbContext*.

Solution

Step 1: Create *User* model (based on your table structure)

```

public class User
{
    public int UserId { get; set; }
    public string Fullname { get; set; }
    public string Description { get; set; }
    public string Password { get; set; } // For practice only; should be hashed
    public string Email { get; set; }
    public string Phone { get; set; }
    public string Address { get; set; }
    public int Status { get; set; }
    public DateTime CreatedDate { get; set; } = DateTime.Now;
    public string UserCode { get; set; }
    public bool IsLocked { get; set; }
    public bool IsDeleted { get; set; }
    public bool IsActive { get; set; }
    public string ActiveCode { get; set; }
    public string Avatar { get; set; }

    // Navigation property for many-to-many
    public ICollection<UserRole> UserRoles { get; set; }
}

```

```
}
```

Step 2: Create *Role* model

```
public class Role
{
    public int RoleId { get; set; }
    public string Name { get; set; }    // e.g., "Admin", "User", "Librarian"
    public string Description { get; set; }

    // Navigation property
    public ICollection<UserRole> UserRoles { get; set; }
}
```

Step 3: Create *UserRole* (join) model

```
public class UserRole
{
    public int UserId { get; set; }
    public User User { get; set; }

    public int RoleId { get; set; }
    public Role Role { get; set; }
}
```

Step 4: Create *AppDbContext*

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) :
    base(options) { }

    public DbSet<User> Users { get; set; }
    public DbSet<Role> Roles { get; set; }
    public DbSet<UserRole> UserRoles { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // Configure many-to-many relationship
        modelBuilder.Entity<UserRole>()
```

```

        .HasKey(ur => new { ur.UserId, ur.RoleId });

modelBuilder.Entity<UserRole>()
    .HasOne(ur => ur.User)
    .WithMany(u => u.UserRoles)
    .HasForeignKey(ur => ur.UserId);

modelBuilder.Entity<UserRole>()
    .HasOne(ur => ur.Role)
    .WithMany(r => r.UserRoles)
    .HasForeignKey(ur => ur.RoleId);

// Optional: seed data
modelBuilder.Entity<Role>().HasData(
    new Role { RoleId = 1, Name = "Admin", Description = "System Administrator" },
    new Role { RoleId = 2, Name = "User", Description = "Normal User" }
);
}
}

```

Exercise 2 – Implement Login with JWT in AuthController

Question

Create a login endpoint `/api/auth/login` that validates user credentials, retrieves all assigned roles, and returns a JWT including those roles in the claims.

Solution

DTO: `LoginRequest.cs`

```

public class LoginRequest
{
    public string Email { get; set; }
    public string Password { get; set; }
}

```

Controller: `AuthController.cs`

```

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase

```

```

{
    private readonly AppDbContext _context;
    private readonly IConfiguration _configuration;

    public AuthController(AppDbContext context, IConfiguration
configuration)
    {
        _context = context;
        _configuration = configuration;
    }

    [HttpPost("login")]
    public IActionResult Login([FromBody] LoginRequest request)
    {
        var user = _context.Users
            .Include(u => u.UserRoles)
            .ThenInclude(ur => ur.Role)
            .FirstOrDefault(u => u.Email == request.Email && u.Password ==
request.Password);

        if (user == null)
            return Unauthorized("Invalid email or password.");

        var token = GenerateJwtToken(user);
        return Ok(new { token });
    }

    private string GenerateJwtToken(User user)
    {
        var roles = user.UserRoles.Select(ur => ur.Role.Name).ToList();

        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.Name, user.Email)
        };

        // Add multiple role claims

```

```

        claims.AddRange(roles.Select(role => new Claim(ClaimTypes.Role,
role)));

        var key = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(_configuration["JwtSettings:SecretKey"]))
;

        var creds = new SigningCredentials(key,
SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: _configuration["JwtSettings:Issuer"],
            audience: _configuration["JwtSettings:Audience"],
            claims: claims,
            expires: DateTime.Now.AddMinutes(60),
            signingCredentials: creds
        );

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}

```

Exercise 3 – Create a Protected Endpoint

Question

Create `/api/users/me` endpoint to display information about the logged-in user, including their roles.
This endpoint must require a valid JWT.

Solution

```

[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    private readonly AppDbContext _context;

    public UsersController(AppDbContext context)
    {
        _context = context;
    }

    [HttpGet("me")]

```

```

[Authorize]
public IActionResult GetMe()
{
    var email = User.Identity?.Name;
    var roles = User.FindAll(ClaimTypes.Role).Select(r => r.Value).ToList();

    return Ok(new
    {
        email,
        roles
    });
}
}

```

Exercise 4 – Role-Based Authorization

Question

Create an admin-only endpoint `/api/admin/dashboard` that requires the "Admin" role.

Solution

```

[ApiController]
[Route("api/[controller]")]
public class AdminController : ControllerBase
{
    [HttpGet("dashboard")]
    [Authorize(Roles = "Admin")]
    public IActionResult AdminDashboard()
    {
        return Ok("Welcome, Admin! Access granted.");
    }
}

```

Exercise 5 – Test with Swagger / Postman

Question

Use Swagger or Postman to test all JWT-protected endpoints.

Solution

Step 1: Login

POST → `/api/auth/login`

```
{
  "email": "admin@example.com",
  "password": "123456"
}
```

Expect a valid token in response.

Step 2: Copy token → use in header:

Authorization: Bearer <your_token>

Step 3: Test `/api/users/me` → shows user info and roles

Step 4: Test `/api/admin/dashboard`

Summary

Exercise	Focus	Key Concepts
1	User–Role Models	Many-to-many relationship with <code>UserRole</code> join table
2	JWT Login	Retrieve roles and issue multi-role JWT
3	Protected Endpoint	<code>[Authorize]</code> with JWT
4	Role Authorization	<code>[Authorize(Roles = "...")]</code>
5	Testing	Token validation and Swagger testing

Exercise 6 – Implement Policy-Based Authorization

Question

Create a custom authorization policy named `"ActiveUserOnly"` that allows access only to users whose account is active (`IsActive == true`).

Apply this policy to an endpoint that returns a list of active users.

Solution

Step 1: Configure the policy in `Program.cs`:

```
builder.Services.AddAuthorization(options =>
{
  options.AddPolicy("ActiveUserOnly", policy =>
    policy.RequireClaim("IsActive", "True"));
});
```

Step 2: When generating the JWT token (in *AuthController*), include a claim for *IsActive*:

```
private string GenerateJwtToken(User user)
{
    var claims = new[]
    {
        new Claim(ClaimTypes.Name, user.Username),
        new Claim(ClaimTypes.Role, user.Role),
        new Claim("IsActive", user.IsActive.ToString())
    };

    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(_configuration["JwtSettings:SecretKey"]));
    var creds = new SigningCredentials(key,
        SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: _configuration["JwtSettings:Issuer"],
        audience: _configuration["JwtSettings:Audience"],
        claims: claims,
        expires: DateTime.Now.AddMinutes(60),
        signingCredentials: creds
    );
    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

Step 3: Protect an endpoint with this policy:

```
[HttpGet("active-users")]
[Authorize(Policy = "ActiveUserOnly")]
public IActionResult GetActiveUsers()
{
    var users = _context.Users.Where(u => u.IsActive).ToList();
    return Ok(users);
}
```

Exercise 7 – Create a Role + Claim Combination Policy
Question

Create a policy called *"AdminOrLibrarian"* that grants access if the user has either the *"Admin"* or *"Librarian"* role.

Apply this policy to the */api/reports/top-borrowed* endpoint.

Solution

Step 1: Configure the policy:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOrLibrarian", policy =>
        policy.RequireRole("Admin", "Librarian"));
});
```

Step 2: Apply it in the controller:

```
[HttpGet("top-borrowed")]
[Authorize(Policy = "AdminOrLibrarian")]
public IActionResult GetTopBorrowedBooks(DateTime fromDate, DateTime
toDate, int top = 10)
{
    var report = _context.Loans
        .Where(l => l.LoanDate >= fromDate && l.LoanDate <= toDate)
        .GroupBy(l => l.BookId)
        .Select(g => new {
            BookId = g.Key,
            Title = _context.Books.FirstOrDefault(b => b.BookId == g.Key).Title,
            TimesBorrowed = g.Count()
        })
        .OrderByDescending(x => x.TimesBorrowed)
        .Take(top)
        .ToList();

    return Ok(report);
}
```

Exercise 8 – Implement a Minimum Membership Duration Policy

Question

Define a policy named *"MinimumMembership"* that allows only users who have been registered for more than **30 days**.

Apply this policy to the endpoint */api/loans/history* so that only long-term users can view their loan history.

Solution

Step 1: Create a custom requirement and handler:

Requirements/MinimumMembershipRequirement.cs

```
public class MinimumMembershipRequirement : IAuthorizationRequirement
{
    public int MinimumDays { get; }
    public MinimumMembershipRequirement(int days) => MinimumDays =
days;
}
```

Handlers/MinimumMembershipHandler.cs

```
public class MinimumMembershipHandler :
AuthorizationHandler<MinimumMembershipRequirement>
{
    private readonly AppDbContext _context;

    public MinimumMembershipHandler(AppDbContext context)
    {
        _context = context;
    }

    protected override Task
HandleRequirementAsync(AuthorizationHandlerContext context,
    MinimumMembershipRequirement requirement)
    {
        var username = context.User.Identity?.Name;
        var user = _context.Users.FirstOrDefault(u => u.Fullname ==
username);

        if (user != null && user.CreatedDate <=
DateTime.Now.AddDays(-requirement.MinimumDays))
        {
            context.Succeed(requirement);
        }

        return Task.CompletedTask;
    }
}
```

Step 2: Register the handler and policy in *Program.cs*:

```
builder.Services.AddSingleton<IAuthorizationHandler,  
MinimumMembershipHandler>();
```

```
builder.Services.AddAuthorization(options =>  
{  
    options.AddPolicy("MinimumMembership",  
        policy => policy.Requirements.Add(new  
MinimumMembershipRequirement(30)));  
});
```

Step 3: Apply the policy:

```
[HttpGet("history")]  
[Authorize(Policy = "MinimumMembership")]  
public IActionResult GetLoanHistory()  
{  
    var username = User.Identity?.Name;  
    var user = _context.Users.FirstOrDefault(u => u.Fullname == username);  
    var loans = _context.Loans.Where(l => l.UserId == user.UserId).ToList();  
    return Ok(loans);  
}
```

Exercise 9 – Implement Policy for Active Category Management

Question

Create a policy *"ManageActiveCategories"* that only allows access to users with the *"Admin"* role **and** the claim *"CanManageCategories" = "True"*.

Use this for the */api/categories* management endpoints.

Solution

Step 1: Configure the policy:

```
builder.Services.AddAuthorization(options =>  
{  
    options.AddPolicy("ManageActiveCategories", policy =>  
        policy.RequireRole("Admin")  
            .RequireClaim("CanManageCategories", "True"));  
});
```

Step 2: Include the claim when generating tokens for admins:

```
if (user.Role == "Admin")  
{
```

```
claims.Add(new Claim("CanManageCategories", "True"));
}
```

Step 3: Apply the policy:

```
[HttpPost]
[Authorize(Policy = "ManageActiveCategories")]
public IActionResult AddCategory([FromBody] Category model)
{
    _context.Categories.Add(model);
    _context.SaveChanges();
    return Ok(model);
}
```

Exercise 10 – Create Policy for Verified Email Users

Question

Implement a policy called *"VerifiedEmailOnly"* that restricts access to certain features (like uploading PDFs or avatars) to users with verified emails (*EmailConfirmed = true*).

Apply it to */api/books/upload*.

Solution

Step 1: Add claim *"EmailVerified"* when generating JWT:

```
new Claim("EmailVerified", user.IsActive && user.IsDeleted == false ?
"True" : "False")
```

Step 2: Configure the policy:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("VerifiedEmailOnly", policy =>
        policy.RequireClaim("EmailVerified", "True"));
});
```

Step 3: Protect upload endpoint:

```
[HttpPost("upload")]
[Authorize(Policy = "VerifiedEmailOnly")]
public async Task<IActionResult> UploadBookFiles(IFormFile pdf, IFormFile
cover)
{
    if (pdf == null || cover == null)
        return BadRequest("Files are required.");
}
```

```
// Handle file upload logic here...
return Ok("Files uploaded successfully!");
}
```

Summary

Exercise	Policy Name	Purpose
6	ActiveUserOnly	Only active users can access
7	AdminOrLibrarian	Allow either Admin or Librarian
8	MinimumMembership	Users with >30 days membership
9	ManageActiveCategories	Admins who can manage categories
10	VerifiedEmailOnly	Only verified email users